

利用區間索引有效處理 DAG 之路徑可達性查詢

余美伶

科立爾數位科技股份有限公司

daisy.yu@corel.com

陳耀輝 陳宇達

國立嘉義大學資訊工程學系

{ychen, s0960413}@mail.ncyu.edu.tw

摘要

在諸多領域中，有許多的應用都是架構在有向無循環圖形上，系統經常必須判斷圖形中的任意兩節點是否有路徑可到達。為了能夠有效地處理路徑可達性的查詢問題，本研究運用了區間索引的編碼方式將遞移封包矩陣中的路徑關係記錄下來，並且發展出一套測試方法能夠有效地判斷節點彼此間的路徑關係為何。主要概念是先對有向無循環圖形任意取出一棵生成樹，並且利用數值區間索引對生成樹做編碼，再將圖形中未走訪到的非樹邊路徑關係記錄在遞移封包矩陣中。若是兩節點間的路徑關係可藉由生成樹的樹邊路徑到達，則僅需比對兩節點的區間索引便可知其包含關係。若是兩節點彼此間要透過樹邊路徑與非樹邊路徑才能到達，則除了比對索引之外還要檢查遞移封包矩陣中相關的非樹邊路徑端點 (Tails 與 Heads) 關係記錄方能知道結果。此方法只需常數時間就可以完成圖形中任意兩節點的路徑關係測試，另外對於圖形結構的更新處理，也只需針對局部的範圍作路徑關係記錄調整。

1 緒論

在有向無循環圖形 (Directed Acyclic Graph, DAG) 中，最常遇到的問題是任意兩節點間的路徑可達性查詢。所謂的路徑可達性 (Reachability) 是指在一個有向無循環圖形中，如果存在一條路徑可連通兩節點，則表示這兩節點彼此之間具路徑可達性。而目前許多領域的應用都是架構在有向無循環圖形上，像是封包排程，類神經網路，和模組化語言等。此外，像一般系統所採用的安全存取管控機制，例如：以角色為基礎的存取控制模組、以任務為基礎的存取控制模型等等，這些安全機制中的存取權限均具有多重繼承的特性，而多重繼承的結構本身就屬於一種有向無循環圖。因此針對此結構的路徑可達性測試提出一套有效地查詢方式成為非常重要的探討議題。

在一個有向無循環圖形 $G = (V, E)$ 中，處理兩節點間是否存在路徑能夠到達對方位置，最簡單且直覺的方法就是尋訪有向無循環圖形上的所有指標路徑。當圖形結構中所有的指標都被走訪過後，便可以得知是否有路徑存在兩節點彼此之間。然而此種處理方式非常耗時，每查詢一次所耗費的時間成本為 $O(|E| + |V|)$ 。為了改

善在查詢時過於耗費時間的問題，我們可以製作遞移封包矩陣 (Transitive Closure Matrix)，將有向無循環圖形中各節點彼此間的所有路徑關係全部記錄在遞移封包矩陣，如此一來當系統在處理路徑可達性查詢時，僅需要比對遞移封包矩陣中相關的路徑關係資訊便可以完成測試，其所花費的時間成本為 $O(1)$ 。雖然這種方法在查詢處理時節省了大量的時間，但是利用遞移封包矩陣來記錄圖形中所有的路徑關係卻會將儲存空間的複雜度提高到 $O(|V|^2)$ 。當有向無循環圖非常大時，此種方法就會變的不適用。由此而知，當路徑可達性查詢效能達到最佳化的同時，將耗費龐大的儲存空間來記錄節點間的路徑關係；相反地當儲存空間複雜度降低時，路徑可達性查詢的效能也會因此而受影響。因此在一個有向無循環圖中，如何能同時達到儲存空間與查詢效能的最佳化，是個常被提出探討的議題。

在本篇論文中，我們將提出一個能夠有效地處理路徑可達性查詢的演算法，並且不需要耗費太多的儲存空間即可記錄整個有向無循環圖形中的路徑關係。本文的文章結構如下，第二節針對目前已被提出的圖形節點路徑關係查詢之相關研究進行探討與比較，第三節描述我們所提之方法以及核心概念，第四節呈現如何藉由改變路徑關係的記錄模式以降低時間成本，第五節主要探討圖形結構更新的問題處理，第六節為本方法之時間與空間複雜度分析，最後第七節為結論與未來研究方向。

2 相關研究

在一個有向無循環圖形 $G = (V, E)$ 中，查詢兩節點間的路徑可達性，最簡單的方法就是走尋有向無循環圖形上的所有指標路徑，便可得知兩節點間是否有路徑可到達。然而此種處理方式非常耗時，每查詢一次所耗費的時間成本為 $O(|E| + |V|)$ 。為了解決此問題目前已有許多針對改善圖形中路徑可達性關係測試的相關研究被提出。

2.1 Bit-Vector 方法

在一個 $G = (V, E)$ 的有向無循環圖形中，對於任意節點的路徑關係測試問題，傳統的解決方式是利用遞移封包矩陣 (Transitive closure matrix)，將圖形中所有節點彼此之間的路徑關

係以 0 和 1 來記錄。然而此方法浪費了太多的儲存空間與矩陣編碼時間，所以 [2] [3] 改進了遞移封包矩陣的缺點，利用“調節”的概念在需要的時候才會採用新增一位元來記錄編碼值，其將整個結構相關性高的部分劃分為同一模組區塊，而後針對模組內部的節點作編碼，最後再將所有模組的編碼值整合。若有節點的編碼值因模組區塊整合後而不具唯一性，則會增加一個新位元以作區隔，如此便能使更多的位元單位能重複使用，做為新群組的編碼值。然而在處理結構劃分模組部分就耗費了 $O(|V|^2)$ 的時間複雜度以及 $O(|VD|)$ 的儲存空間，其中 V 為圖形結構中的節點數， D 則為整個結構的階層數。[9] 提出一套新的演算法來解決路徑關係記錄的儲存問題，其所採用的編碼方式是先採由下而上 (bottom-up)，再由上而下 (top-down) 的方式走訪圖形並給予節點索引，共需要走訪結構兩次方能完成編碼。然而此方法屬於靜態編碼，當圖形結構遇到更新時無法處理局部的節點索引值。[4] [5] 則利用二元索引方式編碼，只有當新增一個節點會跟其他節點編碼值產生矛盾，造成節點的索引值不唯一時，才會選擇一個新的位元來標記。此種索引編碼方式是採用由上而下的走訪方式所產生，因此面對葉節點的更新時，也能做到直接給予新節點新的索引值，而不必影響到其他已編碼完成的節點，但是當要更新處理的節點位於結構越上層的位置時，影響到其他已編碼完成的節點範圍則越大。

2.2 Interval 方法

區間索引編碼是一種快速判斷節點彼此間路徑關係的方法。樹中的任意兩節點是否具有路徑關係存在，可藉由走訪樹時的順序來決定 [7]：在樹 T 上取任意兩節點 X 與 Y ，則節點 X 可透過路徑到達節點 Y ，若且為若當採前序走訪 T 時則 X 將會比 Y 早走訪到，且當採後序走訪 T 時則 X 會比 Y 晚走訪到。[13] 則提出了三種不同的編碼方法，分別為：(1) 以深度優先走訪方式依序給定一數值 (Global Order Encoding)。(2) 由每個節點與相關的兄弟節點依序給定數值 (Local Order Encoding)。(3) 類似方法二，均對每個節點與相關的兄弟節點依序給定數值，不同的是每個節點必須繼承父親節點的數值 (Dewey Encoding)。其編碼時間只需 $O(|V|)$ ，藉由比對前序值，便可得知兩節點間有無路徑關係存在。但當標記值中的每一個位元超過了十位數時就無法判定，所以需要分用分隔符號「 $\cdot$ 」來分開標記中的每個數值，但過多的分隔符號也造成儲存空間的浪費。且此方法套用在有向無循環圖時，單一節點就有可能產生多組的索引值。[15] 的做法則是由左至右依序給葉節點權重值，且最左與最右的葉節點必須分別給予最小值 1 與一給定的最大值 M 。每個非葉節點的權重值，則給予其所有子孫節點裡的最小與最大權重值為表示。最後根節點的權重值必為 $(1, M)$ ，因此節點的路徑關係可

藉由標記的包含關係來判別。

另外 [1] 採後序走訪的方式給予節點一組範圍值作索引，當一個節點僅存在單一來源路徑時，其被標記為 $[\text{index}(u), \text{post}(u)]$ ， post 為節點 u 的後序走訪數值，而 index 則是在圖形中節點 u 可走訪到的節點當中，其後序數值最小的 (即 $\text{index}(u) \leq \text{post}(u)$ 且葉節點的 $\text{index}(u) = \text{post}(u)$)。為避免新增時需要為結構重新編碼，我們可以在索引上留一些間隔，例如使用一個常數 c ，則 $[\text{index}(u), c \cdot \text{post}(u)]$ 。但此方法套用在有向無循環圖形架構時，每個節點的索引編碼值就不一定只有一個了，而是視節點的路徑關係情況而定。[8] 的編碼方式主要是利用所謂的 *sparse terms* 來編碼，且當有節點的更新時能直接給予新加入的節點索引編碼值，而不需要重新將既有的編碼結構作調整，但是缺點是必須花費額外的儲存空間記錄 *sparse term* 所代表的含意，而且人們無法直接透過編碼值判別任意兩節點是否具有繼承關係，必須藉由查詢 *sparse term* 記錄表方可得知該 *sparse term* 的定義為何。[6] 則提出用 2-hop 編碼方式對圖形先進行叢集 (Cluster)，找出所有的連通圖形之後再將各個連通圖利用編碼做結合，當圖形有多個連通子圖時有最佳的表現。而 [11] 則是提出利用 Two-phase 演算法來產生節點的編碼，其演算法主要是結合了區間索引與 2-hop 的編碼方法，先對圖形取數並利用區間索引編碼，再對未走訪到的剩餘圖利用 2-hop 編碼來解決。[14] 則是運用節點上的兩套索引值來輔助路徑關係查詢的判斷。其先對圖形取樹，使得圖形上的路徑分為樹邊與非樹邊兩種。在取樹的同時並且利用前序與後序的走訪方式為節點編碼，產生樹邊路徑索引值。再針對遞移封包表格中的非樹邊路徑進行函式計算，產生另一套用來檢查非樹邊路徑關係的非樹邊路徑索引值。

2.3 其他方法

除了上述的方法之外，[10] 提出了利用 PQ 樹將結構切割分為多個區塊，把相關性高的節點劃分在同一區塊再做編碼，所耗費的儲存空間複雜度為 $O(k|V|)$ ，其中的 k 為圖形所被分割的區塊數。然而當圖形更新時，必須將結構重新劃分區塊後才能再做編碼。對圖形做分割處理的還有 [12]，運用遞移封包矩陣與鏈 (chain) 做結合，而這些由數個節點所組成的鏈能夠透過部份的中介點將不同區塊的小圖串在一起，如此作法能將路徑關係記錄矩陣壓縮至 $O(m|V|)$ 的儲存空間成本，其中的 m 為圖形中能被鏈所含括的數量。

3 利用區間索引查詢路徑關係

針對上節中所討論的問題與限制，我們希望能夠提出一個較佳的解決方法，在編碼時間、儲存空間以及查詢時間中取得一個最佳平衡點。而我們發現當圖形中節點多重路徑關係越

單純，本研究方法的表現成果最好。相對的若是節點間的多重路徑關係非常複雜時，傳統的遞移封包矩陣將是最佳的解決方法。

3.1 生成樹 (Spanning Tree)

3.1.1 數值區間索引

在多重繼承結構中，所謂的繼承關係即意指為在有向無循環圖形中的任意兩節點是否有路徑可到達。而對於圖形節點路徑可達的查詢處理，我們將運用數值區間索引 (Numerical Intervals) 作為路徑關係判斷的輔助工具。針對巢狀式數值區間索引我們作了定義。

定義 1: (巢狀區間索引集合) 令 $R = \{(b_1, e_1), (b_2, e_2), \dots, (b_n, e_n)\}$ 為一區間索引集合，其中的 $b_1, e_1, b_2, e_2, \dots, b_n, e_n$ 為不同的整數值，且 $b_i < e_i, 1 \leq i \leq n$ 。如果對任意的 $e_i < e_j, 1 \leq i, j \leq n$ ，有 $b_j < b_i < e_i < e_j$ 或是 $b_i < e_i < b_j < e_j$ 的關係，則 R 為一巢狀區間索引的集合。而所謂的巢狀區間索引集合，是指在圖形結構上兩節點彼此間如果存在路徑可走訪到，則該節點的索引值具有包含關係。如 $R = \{(1, 18), (2, 7), \dots, (15, 16)\}$ ， $(1, 18)$ 的索引區間範圍包含了 $(2, 7)$ 的索引區間範圍，表示索引為 $(1, 18)$ 的節點到索引為 $(2, 7)$ 的節點具有路徑關係。

引理 1: (巢狀區間索引) 以深度搜尋的方式走訪有向無循環圖形中的 n 個節點並任意取得一棵生成樹。在取樹的同時依走訪的順序給予節點索引值。令節點 v 為圖形中的一個節點，其數值區間索引為 (b_v, e_v) ，其 b_v 是拜訪時進入節點的順序值 (opening)，而 e_v 則為離開節點的順序值 (closing)。 $R = \{(b_v, e_v) \mid 1 \leq v \leq n\}$ 為一巢狀區間索引的集合。

証明: 詳參[16]。 $\square$

因此針對 DAG 圖形，我們以深度搜尋的方式先對 DAG 圖形任意取出一棵生成樹，且根據走訪的順序給予節點區間索引值，如圖 1。

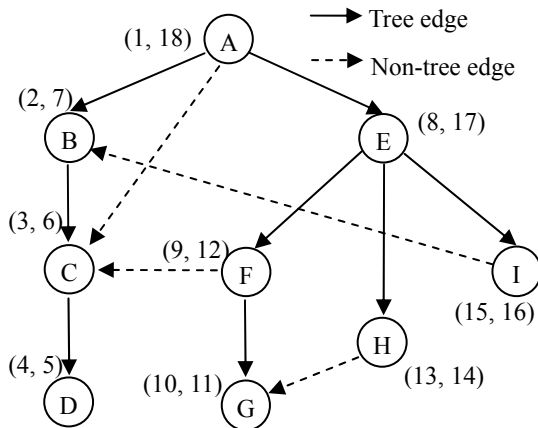


圖 1. 有向無循環圖 (DAG)

3.1.2 樹邊與非樹邊

我們以深度搜尋的方式取得生成樹，並且運

用巢狀式數值區間索引為生成樹上的節點進行編碼。其最大特性是當兩節點彼此之間具有路徑關係存在，則兩節點的數值區間索引則具有包含的關係。若只針對圖形中的生成樹邊作考慮，比對任意兩節點的索引範圍值就可判斷兩點是否有路徑可到達。舉例來說，當節點 P 的範圍值 (a, b) 包含節點 Q 的範圍值 (c, d) 時 (即 $a < c < d < b$)，則表示節點 P 到節點 Q 在生成樹上有路徑可到達。有向無循環圖經過走訪之後，圖形上的所有節點都會被找到並賦予數值區間編碼值，而圖形上的邊則會依據走訪取樹時的路徑選擇分類為樹邊 (tree edges) 與非樹邊 (non-tree edges) 兩種，如圖 1 所示。除了樹邊路徑關係可藉由比較節點的索引值來判斷，對於那些非樹邊的路徑關係，仍可利用節點上的索引值來輔助路徑可達性的查詢處理。

3.2 非樹邊端點 (Heads and Tails)

我們將所有非樹邊的端點 Tail 與 Head 分別以 T 及 H 標記出來，以圖 2 為例，虛線箭頭的那端為 Head，而無箭頭的端點為 Tail。經由圖 1 與圖 2 對照，我們可以知道在有向無循環的圖形中，Head 節點有 B, C, G ，而找到的 Tail 節點有 A, F, H, I 。對於非樹邊上的端點 Head 節點與 Tail 節點，我們做了以下的定義。

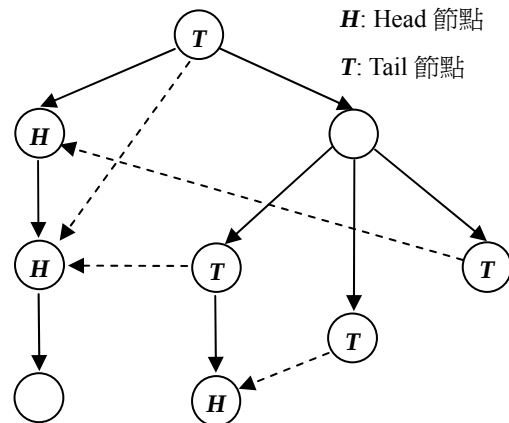


圖 2. 非樹邊端點 (Heads 與 Tails)

定義 2: (非樹邊端點) 令 T 為有向無循環圖 G 上的一棵生成樹。如果有一非樹邊路徑連接 P, Q 兩節點，使得 P 能夠到達 Q ，則節點 P 為該非樹邊路徑的 Tail 節點，而節點 Q 為其 Head 節點。

接著我們將找出與圖形中各節點相關的 Tail 節點與 Head 節點。由圖 2 可以清楚得知圖形上的各節點被哪些 Head 節點包含。這些 Head 節點可視為被包含的節點的代理 Head 節點，而代理 Head 節點的意思就是如果有任何的節點可以經由非樹邊路徑到達該節點的代理 Head 節點，則必然可藉由節點的代理 Head 節點到達該節點。以節點 D 為例，它被 Head 節點 B, C 所包含。同樣地我們也可以清楚地判別圖形上的各節點包含了哪些的 Tail 節點，而這些被包含的

Tail 節點可視為節點的代理 Tail 節點。所謂節點 P 的代理 Tail 節點意思是如果節點 P 能夠經由非樹邊路徑被走訪到圖形中的任一節點，則該節點必然也可以經由節點 P 的某一個代理 Tail 節點走訪到。以節點 E 為例，該節點包含了 Tail 節點 F、H 和 I。因此當圖形中的任意兩節點必須藉由走訪非樹邊路徑才會到達時，我們就可以運用這些非樹邊端點 Heads 與 Tails 的索引值來幫忙檢查，此時兩節點的路徑關係查詢處理在含義上則轉換為兩節點的代理 Tail 節點與代理 Head 節點的路徑關係查詢。若是這些與測試節點相關的 Tail 節點與 Head 節點的索引值彼此間具有包含關係，則表示欲查詢的兩節點具路徑可達性。

我們以節點 E 與節點 D 為例。如圖 1 所示，節點 E 所包含的代理 Tail 節點有 F、H、I，而節點 D 的代理 Head 節點則為 C。而藉由代理 Tail 節點與代理 Head 節點發現，節點 E 可藉由其代理 Tail 節點 F、I，到達節點 D 的代理 Head 節點，進而到達節點 D。然而，若是每次檢查都必須靠著走訪圖形中代理節點的路徑關係才能判斷是非常沒效率的，因此我們將把這些代理 Tail 與 Head 節點彼此間的路徑關係，記錄於遞移封包矩陣中以便日後能夠快速地查詢 Tail 到 Head 間的路徑關係結果。

表 1. 遞移封包矩陣
Head 節點

		B (2, 7)	C (3, 6)	G (10, 11)
Tail 節點	A(1, 18)	1	1	1
	F (9, 12)	0	1	1
	H (13, 14)	0	0	1
	I (15, 16)	1	1	0

3.3 遞移封包矩陣

除了為生成樹編碼，我們也將有向無循環圖形中非樹邊端點 Tail 節點及 Head 節點之間的路徑關係記錄於遞移封包矩陣。在矩陣中，我們利用遞迴的方式將有向無循環圖形中非樹邊端點 Tails 到 Heads 間的所有路徑關係都記錄下來，若是在有向無循環圖中有路徑關係存在則記錄為 1，反之則記錄為 0。此矩陣會將有向無循環圖形中無法僅藉由樹邊方可到達的路徑關係全部記錄下來。因此，我們以些許的空間成本換取系統檢查權限的執行效能，不需要像傳統的遞移封包矩陣那樣浪費儲存空間便可將結構中所有的路徑關係記錄下來。以圖 1 為例，我們以深度搜尋的方式走訪有向無循環圖並取樹，同時用數值區間索引為生成樹上的節點進行編碼(編碼值如圖所示)而由 Tail 節點與 Head 節點所構成的遞移封包矩陣如表 1。而後由圖 3

的演算法我們可以產生非樹邊遞移封包矩陣，而矩陣中 Tail 與 Head 節點彼此間的路徑關係，不論是僅只藉由非樹邊路徑可到達，或是透過樹邊與非樹邊路徑到達的路徑關係情形，均會完整地記錄於非樹邊遞移封包矩陣中。

```

Transclosure (int adjmat[2m][2m], int
path[2m][2m])
01.  for (i = 1; i ≤ 2m; i++) {
02.      for (j = 1; i ≤ 2m; i++) {
03.          path[i][j] = adjmat[i][j];
04.          if (path[j][0].pair.left_value ≤
              path[0][i].pair.left_value ≤
              path[0][i].pair.right_value ≤
              path[j][0].pair.right_value ≤)
05.              path[i][j] = 1; } }
06.  for (k = 1; k ≤ 2m; k++) {
07.      for (i = 1; i ≤ 2m; i++) {
08.          for (j = 1; i ≤ 2m; j++) {
09.              path[i][j] = path[i][j] || path[i][k] &&
              path[k][j];
10.  } } }

```

圖 3. 非樹邊路徑遞移封包矩陣演算法

```

BasicReachabilityTest (v, w)
01.  if ( $b_v < b_w < e_w < e_v$ )
02.      there is a path from v to w in the spanning
      tree;
03.  else if (any of v's representing tail nodes is
      connected to w's representing head)
04.      there is a path from v to w in G;

```

圖 4. 路徑可達性測試演算法

在演算法中對於樹邊路徑的關係則是採用比對節點的區間索引值來判斷。而節點彼此間的路徑可達性查詢測試演算法則如圖 4 所示。任意兩節點是否有路徑可到達的查詢處理主要分為兩部份，一部份是檢查相關的樹邊路徑關係，另一部分則是檢查相關的非樹邊路徑關係。在檢查相關的樹邊路徑關係時，由比對節點上的索引範圍值，即可得知節點彼此間是否有樹邊路徑關係存在，如果兩節點的索引範圍值具有包含關係回傳結果，反之則必須繼續檢查節點間是否有非樹邊路徑關係存在。當我們檢查非樹邊路徑關係時，節點路徑關係查詢問題則轉換為代理 Tail 節點與代理 Head 節點的路徑關係查詢。如果節點的代理 Tail 節點與另一節點的代理 Head 節點具有路徑關係，則表示節點可經由到代理 Tail 節點走訪另一節點的代理 Head 節點，進而到達另一節點。故若代理 Tail 節點與代理 Head 節點間存在路徑關係，則可判定兩節點間必然存在一條以上的路徑可到達。

因此若要判斷在有向無循環圖中的任意兩節點是否有路徑可達性關係，只要檢查兩節點的數值區間索引值與遞移封包矩陣中相關的 Tails 與 Heads 路徑關係記錄，便能快速得到結果。

以圖 1 中的節點 I 與節點 C 為例，節點 I 的索引值為 (15, 16) 而節點 C 的索引值為 (3, 6)。

首先對兩節點的數值區間索引範圍值判別是否具有包含關係。由於 (15, 16) 並不包含 (3, 6)，所以節點 I 無法僅經由生成樹樹邊路徑到達節點 C。因此繼續檢查非樹邊的路徑關係，由圖 2 可知節點 I 為自己本身可經由樹邊到達的 Tail 節點，而符合的 Head 節點有節點 C 自身以及節點 B。接著檢查表 1 中的 I (15, 16) 與 B (2, 7) 所對應到的矩陣位置，元素值為 1 則表示節點 I 可經由節點 B 到達節點 C。一旦檢查到兩者有符合關係時，就判別兩節點之間具有路徑可達，並會停止後續的檢查動作。

4 有效地記錄與比對路徑關係

產生上述的編碼及記錄矩陣路徑關係只需花費 $O(|V_m|^3 + |V|\log|V|)$ 的時間成本與 $O(|V_m|^2 + |V|)$ 的空間成本，其中 $|V_m|$ 是指非樹邊的端點數， $|V|$ 是圖形的節點數。接續我們將提出如何僅透過一次的比對動作，就能在常數時間內完成多筆路徑關係記錄值的檢查。

4.1 代理節點範圍值

雖然節點非樹邊路徑關係可以藉由 Tail 節點與 Head 節點作為輔助，使其得知非樹邊路徑遞移封包矩陣中需要搜尋檢查的區塊位置，然而這樣仍無法在常數的時間內完成檢查並判別出節點的路徑關係。針對此問題我們提出了改善方法，希望藉由找尋並記錄各節點的代理 Tail 與 Head 範圍值，方便之後能夠快速地獲得與節點相關的代理節點集合，進而輔助查詢非樹邊路徑遞移封包矩陣中的相關區塊。首先列出非

樹邊路徑上所有的 Tail 和 Head 數值範圍，並依據 Tail 與 Head 的編碼索引值進行排序動作，如圖 5 所示。再將圖形上各節點的索引值分別與 Head 和 Tail 索引做對應以取得各節點的 Tail 範圍值與 Head 範圍值，如表 2 和表 3。

定義 3: (代理 Head 節點與代理 Head 區間範圍) 令 T 為有向無循環圖 G 上的一棵生成樹。若是節點 v 為任一 Head 節點且其索引值 (b_v, e_v) 擁有最大的 b_v 值與最小的 e_v 值，使得 $b_w \leq b_v < e_w \leq e_v$ ，其中的 (b_v, e_v) 和 (b_w, e_w) 為取樹時給予節點 v 和節點 w 的數值區間索引值。這樣我們就可以判定節點 v 為節點 w 的代理 Head 節點。此外， (b_v, e_v) 亦為節點 w 的代理 Head 區間範圍值。

定義 4: (代理 Tail 節點與代理 Tail 區間範圍) 令 T 為有向無循環圖 G 上的生成樹。若是節點 v 為任一 Tail 節點且其索引值為 (b_v, e_v) 且 $b_w \leq b_v < e_v \leq e_w$ ，其中的 (b_v, e_v) 和 (b_w, e_w) 為取樹時給予節點 v 和節點 w 的數值區間索引值，則節點 v 為節點 w 的代理 Tail 節點。而節點 w 的代理 Tail 區間範圍為 (b_p, e_q) ，其中的 b_p 為節點 w 所有代理 Tail 節點索引區間值的最小起始值，而 e_q 則為節點 w 所有代理 Tail 節點索引區間值的最大終止值。

根據定義 3 可知代理 Tail 範圍值就是將該節點包含的所有 Tail 索引值找出來，並在這 Tail 索引集合中找出最小與最大值作為代理 Tail 範圍的起始值與終止值，如此只要是該節點的代理 Tail 節點其區間索引值必定落於代理 Tail 範圍值的區段內。而根據定義 4，我們則以與節點最為鄰近的 Head 節點區間索引值為代理 Head 範圍值。

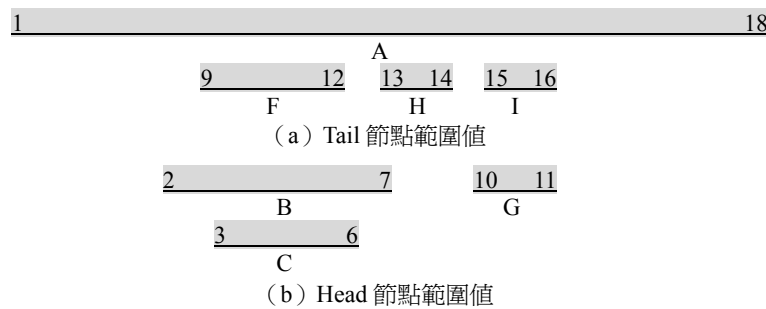


圖 5. 非樹邊端點範圍值

表 2. 代理的 Tail 節點範圍值

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
begin	1	9	9	9	9	9	9	9	9	13	13	13	13	15	15	+	+	+
end	-	-	-	-	-	-	-	-	-	-	-	12	12	14	14	16	16	18

表 3. 代理的節點 Head 範圍值

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
begin		2	3	3	3	3	2			10	10							
end		7	6	6	6	6	7			11	11							

```

FindInsideInterval (R, S)
01. /* R = {(bi, ei) | 1 ≤ i ≤ n} */
02. /* S = {(bj, ej) | (bj, ej) is in R, 1 ≤ j ≤ m} */
03. Sort values in R and store them in r[] which length is equal to 2|V| in G;
04. Sort begin values in S and store them in b[];
05. Sort end values in S and store them in e[];
06. for (i = 1; i ≤ 2n; i++) {
07.   TailRange[1][i] = +; /* the positive infinity of begin values */
08.   TailRange[2][i] = -; /* the negative infinity of end values */
09. begin = 1;
10. for (i = 1; i ≤ 2n; i++) {
11.   if (begin ≤ m) {
12.     if (r[i] < b[begin])
13.       TailRange[1][i] = b[begin];
14.     else if (r[i] = b[begin]) {
15.       TailRange[1][i] = b[begin];
16.       begin++;
17.     }
18.   }
19.   else /* without value in b[] when begin is bigger than m */
20.     break;
21. end = m;
22. for (i = 2n; i ≥ 1; i--) {
23.   if (end ≥ 1) {
24.     if (r[i] > e[end])
25.       TailRange[2][i] = e[end];
26.     else if (r[i] = e[end]) {
27.       TailRange[2][i] = e[end];
28.       end--;
29.     }
30.   }
31.   else /* without value in e[] when end is smaller than 1 */
32.     break;
33. }
34. /* Results are in TailRange[1][i] and TailRange[2][i] */

```

圖 6. 找尋節點的代理 Tail 範圍值演算法

引理 2：(判斷是否存在內部區間) 在一個 $S = \{(b_1, e_1), (b_2, e_2), \dots, (b_m, e_m)\}$ 的一巢狀區間索引集合中，對任意的 $b_i, e_j, 1 \leq i, j \leq m$ ，如果 $b_i < e_j$ 則存在一個 $k, 1 \leq k \leq m$ ，使得 $b_i \leq b_k < e_k \leq e_j$ 。

證明：如果 $b_i < e_j$ 則將會有三種情形可能發生：(1) $b_j < b_i < e_i < e_j$ ，(2) $b_i < b_j < e_j < e_i$ ，以及 (3) $b_i < e_i < b_j < e_j$ 。不論是哪一種情形發生，必然至少存在一個 k 值使得 $b_i \leq b_k < e_k \leq e_j$ 。□

定理 1：(找出代理 Tail 區間範圍) 令 R 為一巢狀區間索引集合，而且 S 為 R 的子集合。對 R 中任意起始值 b 及終止值 e ，可以藉由演算法 FindInsideInterval(R, S) 找到在 S 中是否有區間索引存在於 b 及 e 的範圍內。

證明：詳參[16]。□

藉由引理 2 和定理 1 的應用，我們可以找到各節點的代理 Tail 範圍值。而代理 Tail 與 Head 範圍值的製作演算法如圖 6 與圖 7 所示。圖 6 找尋代理 Tail 範圍值的做法主要是先將所有的代理 Tail 節點排序後，分別將各節點的起始索引

值與終止索引值和排序過後的 Tail 節點起始索引值與終止索引值作比對後，便可以找到各節點的代理 Tail 範圍值。而代理 Head 範圍值與代理 Tail 範圍值的做法相似，不過多了一個堆疊做輔助來暫存當下符合的 Head 節點，因為我們只記錄最鄰近的代理 Head 節點的範圍值。

以節點 A 為例，查表 2 便可得到 Tail 範圍值的 begin 值為 1，end 值為 18，這表示節點 A 包含了數值範圍為 (9, 12)、(13, 14)、(16, 17) 的三個 Tail 節點 (F, G, D) 及其對應的非樹邊路徑關係。若以節點 B 為例，查表 2 的 Tail pair 而得到 Begin 值記錄為“9”而 End 值記錄為“-”，所以找不到其所包含的非樹邊路徑關係。而 Head pair 如表 3，因為必須找到最鄰近的 Head pair，因此只要找到比節點本身範圍值再大一點或等於，且具有包含關係即可。以節點 D 為例，其編碼值為 (4, 5)，雖然 Head 節點 B, C 及 D 的範圍值 (2, 7)、(3, 6) 和 (4, 5) 均有包含節點 D 的範圍值，但只取最小的範圍，所以找到 Head

```

FindOutsideInterval (R, S)
01. /* R = {(bi, ei) | 1 ≤ i ≤ n} */
02. /* S = {(bj, ej) | (bj, ej) is in R, 1 ≤ j ≤ m} */
03. Stack ← {-}
04. Sort values in R and store them in r[] which length is equal to 2|V| in G;
05. Sort begin values in S and store the sorted pair's begin and end values in p[1][] and p[2][], respectively, e.g.
   p[1][4] is the 4th smallest begin value and p[2][4] is its corresponding end value;
06. for (i = 1; i ≤ 2n; i++) {
07.   HeadRange[1][i] = null; /* the initiation of begin values */
08.   HeadRange[2][i] = null; /* the initiation of end values */
09.   j = 1;
10.   for (i = 1; i ≤ 2n; i++) {
11.     if (j ≤ m) {
12.       if (Stack = {-}) {
13.         if (p[1][j] = r[i] && r[i] ≤ p[2][j]) {
14.           HeadRange[1][i] = p[1][j];
15.           HeadRange[2][j] = p[2][j];
16.           Stack.push(pair(p[1][j], p[2][j]));
17.           j++; }
18.         else {
19.           if (p[1][j] = r[i] && r[i] ≤ p[2][j]) {
20.             HeadRange[1][i] = p[1][j];
21.             HeadRange[2][j] = p[2][j];
22.             Stack.push(pair(p[1][j], p[2][j]));
23.             j++; }
24.           else if (p[1][j] < r[i]) {
25.             if (Stack.top(pair.left_value) < r[i]) {
26.               HeadRange[1][i] = Stack.top(pair.left_value);
27.               HeadRange[2][i] = Stack.top(pair.right_value); }
28.             else if (Stack.top(pair.left_value) = r[i]) {
29.               HeadRange[1][i] = Stack.top(pair.left_value);
30.               HeadRange[2][i] = Stack.top(pair.right_value);
31.               Stack.pop(); } } }
32.         else /* without value in b[][] when j is bigger than m */
33.           break;
34.       }
35.   } /* Results are in HeadRange[1][i] and HeadRange[2][i] */

```

圖 7. 找尋節點的代理 Head 範圍值演算法

的 Begin 值為 3，而 End 值為 6。

4.2 矩陣記錄

雖然查詢節點非樹邊路徑關係，已經有了代理 Tail 範圍值與代理 Head 範圍值作為輔助。但為了能夠在 $O(1)$ 的時間內完成圖形中任意兩節點的路徑關係查詢，而且整個方法的儲存空間限制在 $O(|V_{nt}|^2 + |V|)$ 複雜度的條件之下，我們將面臨下面問題：若是要比對的非樹邊路徑關係記錄值超過一個以上，如何能夠在 $O(1)$ 的時間內就能馬上判斷結果，而不需要經由檢查個別記錄才能得知結果？

為了解決上述提及的問題，我們將再一次運用數值範圍索引的概念來避免檢查一個個的記錄值，而能在 $O(1)$ 的時間內一次對多個路徑關係記錄值作判斷。由於節點的代理 Head 範圍每次所找出的範圍值為數值範圍最小的 Head 節點索引，所以在路徑關係矩陣中不會有多個數值範圍區段要檢查，而節點的代理 Tail 範圍與代理 Head 範圍相反，其所找出的範圍為被節點包含的所有 Tail 範圍區段，如此一來將會有多個區段的 Tail 數值索引必須檢查，所以我們改變矩

陣中 Tail 節點和 Head 節點路徑關係的記錄型態，不再是以 0 與 1 作各節點間的路徑關係記錄，而是在矩陣中以表格的型態（Head node's Tail ranges Matrix，HTM）來記錄 Tail 節點和 Head 節點彼此之間的路徑關係。

表 4. Head 節點的 HTM

Head B (2, 7)								
Range	1	9	12	13	14	15	16	18
Begin	1	15	15	15	15	15	+	+
End	-	-	-	-	-	-	16	18

Head C (3, 6)								
Range	1	9	12	13	14	15	16	18
Begin	1	9	15	15	15	15	+	+
End	-	-	12	12	12	12	16	18

Head G (10, 11)								
Range	1	9	12	13	14	15	16	18
Begin	1	9	13	13	+	+	+	+
End	-	-	12	12	14	14	14	18

在 HTM 中，每一個 Tail 節點和 Head 節點的對應位置會有 Begin 與 End 兩個記錄值，而 Begin 與 End 記錄值的產生則是再一次運用定理 1 的概念。若是當 Tail 節點 Head 到節點有路徑關係存在時，則將會把該 Tail 節點的數值範圍索引值記錄於 Head 節點與 Tail 節點所對應到的 HTM 位置中，反之則以“+”和“-”作記錄，其結果如表 4 所示。以 Head 節點 B 為例，其在表 1 非樹邊路徑遞移封包矩陣中與 Tail 節點 A(1, 18) 和 Tail 節點 I(15, 16) 有路徑關係，而與 Tail 節點 F(9, 12) 和 Tail 節點 H(13, 14) 沒有路徑關係。這意謂著在 Tail 節點集合的索引範圍中，有兩個索引值區段 (1, 18) 以及 (15, 16) 包含了 Head 節點 B 的索引範圍，亦即經由 Tail 節點 (1, 18) 和 (15, 16) 兩索引區段可以到達 Head 節點 (2, 7)。因此在 HTM 中記錄 Head 節點 B 的路徑關係時，我們將所有的 Tail 節點索引值對應到這兩個區段的索引值進行範圍值的比對，再次運用數值區間索引的特性重新計算 Head 節點 B 與各 Tail 節點的路徑關係，而且以新取得的數值索引範圍值 (Begin 值和 End 值) 做為新的路徑關係記錄值。

我們再次應用定義 1，將取得的 Begin 值與 End 值做下列的條件判斷：End 值必須大於等於 Begin 值。因為若是 End 值大於等於 Begin 值，則表示在檢查的索引範圍區段中存在一條以上的路徑可以到達該 Head 節點，所以就能得知查詢的兩節點間是否可以經由非樹邊路徑到達彼此。假設我們要判斷 Head 節點 C 與 Tail 節點 H(13, 14) 是否有路徑關係，則我們只需要根據表 4 找到 HTM 中 Head 節點 C 的部分，找到範圍索引值 13 的 Begin 值 15 和範圍索引值為 14 的 End 值 12，並判別 End 值是否大於等於 Begin 值。若 End 值大於等於 Begin 值表示該 Head 節點與該 Tail 節點有路徑關係，反之若取得的 End 值小於 Begin 值則表示此區段的數值範圍是不合法，也就是說該 Head 節點與該 Tail 節點間不存在任何的路徑關係。由本例 End 值 12 小於 Begin 值 15 可知沒有存在任何一條路徑可由 Tail 節點 H 抵達與 Head 節點 C。

定理 2: (路徑可達性測試結果之正確性) 令 R 為一巢狀區間索引集合，而 v, w 為 R 中的任意兩節點。根據演算法 BasicReachabilityTest (v, w) 則可以正確地判斷出節點 v 與節點 w 之間是否有路徑關係存在。

証明：詳參[16]。 □

而據定理 2，可以保證我們所提出的方法在測試路徑可達性查詢時所產生的結果必定是正確的。最後我們將舉例完整說明本研究方法是如執行路徑關係查詢測試的。假設我們要判別節點 E 與節點 D 兩節點間是否有路徑可到達。首先我們先比對兩節點的索引值，節點 E 的索引值為 (8, 17) 而節點 D 的索引值為 (4, 5)。由於索引值 (8, 17) 的範圍並沒有包含到索引值 (4, 5)，所以節點 E 無法只經由樹邊路徑走訪到節點 D，我們將繼續下一步的檢查動作。對照到表 2 取得節點 E 的代理 Tail 範圍值為 (9, 16)，並且也對照到表 3 取得節點 D 的代理 Head

範圍值為 (3, 6)。此時我們再將取得的兩組節點代理範圍值與表 4 作對應，找到 HTM 中的 Head 節點 C (3, 6) 的路徑關係記錄表。根據 Tail 節點 E 的索引值 (9, 16) 取得範圍索引值 9 的 Begin 值為 9，以及範圍索引值 16 的 End 值為 16。因為 16 大於 9，所以我們可以得知查詢的結果為至少存在一條路徑使得節點 E 能夠到達節點 D。

5 圖形結構的更新處理

由於我們採以數值編碼的方式作為路徑關係判斷的輔助工具，因此必然會面臨到索引值更新的問題。而更新的處理方法主要可分為節點的新增及刪除和邊的新增及刪除這四大類情形。

5.1 新增處理

節點新增部分已有許多數值範圍索引更新方法被提出，我們只要採用其中的任一種方法處理即可，而邊新增又可分樹邊更新與非樹邊更新兩種。樹邊的新增處理方式同新增節點的處理方式，而非樹邊的更新則可視為多重繼承的路徑關係處理，只須調整 HTM 中的相關路徑記錄值。

5.2 刪除處理

而刪除部分的處理則是比較複雜。若是刪除節點，相較於新增處理，刪除處理的部分則是比較複雜。因為當有一節點被刪除時，我們必須需要將該刪除節點底下的所有子孫節點與該刪除節點的父節點重新建立新的路徑關係，並且調整 HTM 中的相關記錄值。因此，該刪除節點的子孫節點與其父節點原有的祖孫關係則變為父子關係。為了能夠保有原來的樹狀索引結構不被破壞，並且能夠維護結構的刪除處理需求提出了一個新的解決辦法，我們把刪除動作虛擬化，把刪除的對象作以記號方便日後檢查識別，而非真正的進行刪除動作。我們將產生一個長度為 $2|V|$ 的二維陣列，並且再次利用數值索引範圍包含的概念，將刪除節點的索引值與矩陣中的數值索引值路徑關係記錄於陣列中。而刪除節點索引值與陣列中的數值索引比對與非樹邊端點 Head 範圍值的製作方式相似。若是節點的索引值介於刪除節點的索引值範圍內，則在陣列中將會將刪除節點的索引數值記錄於各節點的索引位置。接著檢查刪除的節點是否存在於 Tail 節點集合或是 Head 節點集合中。如果刪除的節點為 Tail 節點或是 Head 節點時，則必須調整遞移封包矩陣中的記錄值與 HTM 中的相關路徑關係資訊。如果當已被刪除的節點位置又有新的節點加入時，則給予新節點新的區間索引值，而不是給予舊有刪除節點的索引值。因為新加入的節點不一定會完全地繼承已被刪除節點的所有子樹與路徑關係。然而當有一連串頻繁地新增與刪除動作時，必定會造成

節點的索引值過於碎散，甚至使得非樹邊路徑遞移封包矩陣與 HTM 中的記錄過於雜亂。因此在這種情況之下，則會建議使用者重新執行我們所提出的方法，產生新的代理區間範圍值、非樹邊路徑遞移封包矩陣以及 HTM，如此一來也能釋放掉過多且不需要的刪除資訊。

樹邊路徑刪除的處理方法與節點刪除相似，將樹邊路徑所到達的節點範圍值作記號，並且與陣列中各節點的索引值做判別檢查。不同的是刪除樹邊路徑在調整遞移封包矩陣中的記錄值與 HTM 中的相關路徑資訊時，只能將予刪除的樹邊路徑有關的記錄做調整，而相關的非樹邊路徑關係必須繼續保留在遞移封包矩陣中。原因是即便是刪除了樹邊路徑，若是節點未被刪除，仍可藉由非樹邊路徑走訪到或是藉由非樹邊路徑到達其他節點。因此我們只需要調整會透過刪除的樹邊到達該節點的路徑記錄，對於該節點可透過樹邊路徑到達其他節點，或是與該節點相關的非樹邊路徑關係記錄均採用不變動的處理方式。當非樹邊被刪除時，我們只需重新調整遞移封包矩陣中的路徑記錄值與 HTM 中的相關路徑資訊，而不需要再去刪除記錄表格與其他的索引值作比對動作，因為刪除非樹邊路徑並不會影響到樹上的索引結構，因此刪除記錄表格主要是針對會刪除動作會影響樹邊路徑關係而作的應對處理。

6 時間與空間複雜度分析

本研究方法運用數值區間索引與遞移封包矩陣作為圖形節點路徑關係判別的輔助工具。前置取樹編碼以及記錄所有非樹邊路徑關係花費了 $O(|V_{nl}|^3 + |V|\log|V|)$ 時間複雜度與 $O(|V_{nl}|^2 + |V|)$ 空間複雜度，其中的 $|V_{nl}|$ 為圖形中的非樹邊端點數量， $|V|$ 則是指圖形中的節點數。而對於圖形中任意節點間的路徑關係查詢，也只需要 $O(1)$ 的判別檢查時間。

6.1 圖形取樹並編碼

當我們取得一個有向無循環圖 $G = (V, E)$ 時，其中 V 為圖形中的節點集合，而 E 為圖形中的邊集合。一開始會先對圖形進行取樹編碼動作。在取出生成樹前，必須先決定圖形中的一點作為生成樹的根節點。針對找尋圖形中的根節點這一部分，我們可以採用拓樸排序演算法（Topological Sort）來決定生成樹的根節點，而拓樸排序演算的時間複雜度為 $O(|E| + |V|)$ 。

決定出生成樹的根節點之後，開始根據圖形上的路徑走訪結構並取得生成樹。走訪取樹的方式並沒有受限於任何特定的方法，而在本方法是採用深度搜尋的方式走訪取樹，並且在走訪取樹的同時給予節點索引編碼值。因為必須將圖形上所有的節點都走訪過，所以花費了 $O(|V|)$ 的時間複雜度。另外，由於結構取樹走訪後每個節點均有一個數值區間索引，因此必須要 $O(|V|)$ 的儲存空間來記錄這些索引值。

6.2 遞移封包矩陣製作

當生成樹完成編碼時，圖形上的路徑將會分成兩大類，一類是樹邊路徑而另一類則為非樹邊路徑。然而樹邊路徑關係可藉由生成樹上的節點索引來判別，可是若只有藉著樹上的節點索引資訊來判斷節點彼此間是否能透過這些非樹邊路徑到達，是無法正確判斷出結果的，因此我們將非樹邊路徑上的端點 Tail 與 Head 記錄下來並利用這些端點產生一個遞移封包矩陣。

而非樹邊路徑關係矩陣需要 $|V_{nl}| \times |V_{nl}|$ 的儲存空間來記錄非樹邊路徑上各端點彼此間的路徑關係，所以該矩陣的空間複雜度為 $O(|V_{nl}|^2)$ 。此外，在製作非樹邊路徑遞移封包矩陣的過程中，主要是利用遞迴的方式找出矩陣中所有 Tail 與 Head 端點彼此間的路徑關係，是故該矩陣的產生也需要花費 $O(|V_{nl}|^3)$ 的時間成本。

6.3 代理節點範圍值

非樹邊路徑上的兩端點 Tail 節點與 Head 節點，除了可以用來製作非樹邊路徑遞移封包矩陣之外，尚能輔助節點使其知道要搜尋的相關非樹邊路徑遞移封包矩陣區塊位置為何。但是僅是藉由一個個比對節點與非樹邊路徑端點的索引值，以獲得非樹邊路徑遞移封包矩陣中相關的路徑記錄區塊，仍然無法達到能夠非常快速地檢查並判別出節點間的路徑關係。因此我們利用了些許的空間以換取時間，建立兩個長度各為 $2|V|$ 的表格來記錄各個節點的代理 Tail 範圍值與代理 Head 範圍值。因為圖 6 和圖 7 的演算法中必須先將各節點、Tail 節點與 Head 節點依據索引值排序後，才能獲得各節點的代理 Tail 範圍值和 Head 範圍值，因此其時間複雜度均為 $O(|V|\log|V| + |V_{nl}|\log|V_{nl}|)$ 。而在兩個演算法的處理過程中，必須將各節點、Tail 節點與 Head 節點的索引值記錄下來，所以其空間複雜度則為 $O(|V| + |V_{nl}|)$ 。另外在圖 7 演算法中有利用一堆疊來暫存 Head 節點，以輔助節點找尋代理 Head 範圍值，其堆疊結構所耗的空間複雜度為 $O(|V_{nl}|)$ 。

6.4 HTM 矩陣製作

為了路徑關係查詢能夠在 $O(1)$ 的時間內完成，就必須避免一個個檢查相關記錄的方式。針對此問題，本研究提出新的遞移封包矩陣記錄型態，以表格的方式（Head node's Tail ranges Matrix, HTM）來記錄 Tail 節點和 Head 節點的路徑關係。此部分所耗費的儲存空間成本仍維持在 $O(|V_{nl}|^2)$ 的空間複雜度，而製作新矩陣必須走訪原有的遞移封包矩陣之後才會產生，因此所花費的時間成本則為 $O(|V_{nl}|^2)$ 的時間複雜度。

6.5 DAG 之路徑可達性查詢

根據定理 3 則可得知，採用我們所提出的方法檢查 DAG 中任意兩節點的路徑關係，只要查

表 5. 記錄與比對路徑可達關係之方法比較表

	儲存空間	比對時間	更新處理
Transitive closure	$O(V ^2)$	$O(1)$	為所有節點索引值新增新的位元
Ait-Kaci [2]	$O(V ^2)$	$O(V)$	無法處理更新動作，必須重新編碼
Ait-Kaci [3]	$O(VD)$ (D 指圖形結構的階層數)	$O(V)$	無法處理更新動作，必須重新編碼
Caseau [4]	$O(V ^2)$	$O(V)$	僅受限於處理葉節點的更新，若是非葉節點更新，則必須重新編碼
Agrawal [1]	視多重繼承路徑分布的位置決定	視節點編碼值個數與長度而定	針對更新的部分作局部的索引調整
Dewey [13]	$O((M+1) \times VD)$ (M 指非樹邊的數量， D 指圖形結構的階層數)	視節點編碼值個數與長度而定	針對更新的部分作局部範圍的調整
Gil [10]	$O(k V)$ (k 為圖形分割的區塊數)	視節點編碼值個數而定	無法處理更新動作，必須整個結構重新編碼
Wang [14]	$O(V_{nl} ^2 + V)$	$O(1)$	無法處理更新動作，必須整個結構重新編碼
本研究方法	$O(V_{nl} ^2 + V)$	$O(1)$	視更新的對象與情形作局部的路徑記錄調整

詢前述處理步驟所建立的表格記錄便能夠在 $O(1)$ 的時間複雜度就可以完成路徑可達性的查詢測試。

定理 3:(常數時間完成路徑可達性測試) 令 R 為一巢狀區間索引集合，而且 v, w 為 R 中的兩節點。根據演算法 BasicReachabilityTest (v, w) 則可以在常數時間內完成判斷出節點 v 與節點 w 之間是否有路徑關係存在。

証明：詳參[16]。

□

6.6 更新處理

6.6.1 新增處理

新增的對象可分為節點、樹邊路徑與非樹邊路徑。首先是節點的新增，因為此一情形僅需要找到與新增節點相鄰節點的索引值，並取得適當的索引值配予給新增的節點，接著再將新節點的代理區間範圍值找到並填入代理的 Tail 與 Head 範圍表格中，所以時間成本為 $O(|V| + |V_{nl}|)$ 。而樹邊路徑的新增處理與新增節點的處理是視為一個完整的節點新增動作，所以新增樹邊路徑與新增節點所花費的時間成本和空間成本與新增節點的情形是算相同的。

雖然非樹邊路徑的新增處理分成四種情形，但是大多數的情形都必須去調整非樹邊路徑遞移封包矩陣中相關的路徑關係記錄，以及 HTM 中的相關路徑索引資訊，另外，與新增非樹邊路徑相關的代理節點 Tail 與 Head 範圍值也必須作調整，所以整個新增處理的時間複雜度為 $O(|V_{nl}|^3 + \log|V|)$ 。

6.6.2 刪除處理

同樣地，刪除的對象也可分為節點、樹邊路徑與非樹邊路徑三種。當有樹邊路徑刪除時，會將刪除樹邊路徑所到達的節點索引值作記號，並與刪除表格中各個節點的索引值作數值範圍比對，將受到刪除樹邊路徑影響的索引位置記錄該刪除樹邊路徑所到達的節點索引數值。而為了處理刪除動作而建立的刪除表格，因為記錄了受刪除處理而影響的索引值，花費了空間複雜度 $O(|V|)$ 及時間複雜度 $O(|V|)$ 。

節點刪除的處理則與樹邊路徑刪除的處理方式相似，不同的是在調整非樹邊路徑遞移封包矩陣中相關的路徑關係記錄中，當節點刪除時，所有相關的樹邊路徑與非樹邊路徑記錄都要調整，而當樹邊路徑刪除時，非樹邊路徑記錄則不需要變更。最後再將調整過後的非樹邊路徑遞移封包矩陣轉換為 HTM 中的相關路徑索引資訊，所以整個的時間複雜度包含了刪除表格的處理，以及調整矩陣與 HTM 中的相關記錄，總共花費了 $O(|V_{nl}|^3)$ 的時間。而刪除非樹邊路徑的處理方式，僅需調整非樹邊路徑遞移封包矩陣中的相關路徑記錄，以及在 HTM 中相關代理節點的索引資訊，不需要變更原有刪除表格中的索引記錄值。因此非樹邊路徑的刪除處理只需要 $O(|V_{nl}|^3)$ 的時間複雜度就可以達成。

最後我們將論文中所提出的方法的空間複雜度與時間複雜度，和其他已提出的相關研究相比較，其結果如表 5 所示。

7 結論與未來研究

本研究主要是針對非樹邊路徑數量較少的應用情形，提出一個新的路徑關係記錄方法，並且能夠有效地完成路徑可達性的查詢。運用數值範圍索引編碼與非樹邊遞移封包矩陣中的路徑關係記錄，來檢查圖形中任意兩節點間的路徑可達性。在非樹邊路徑端點數甚小於圖形的總節點數時，整個時間與空間的效能改善結果最佳。而整個編碼時間複雜度為 $O(|V_{nl}|^3 + |V|\log|V|)$ ，儲存空間則為 $O(|V_{nl}|^2 + |V|)$ 的複雜度，其中的 $|V_{nl}|$ 指非樹邊的端點數， $|V|$ 是圖形中的節點數。此外利用略微的空間成本，換取在常數時間內完成圖形中節點彼此間的路徑關係檢查動作。然而當非樹邊路徑的數量非常多的情形時，最好的方法仍是傳統的遞移封包矩陣，不僅耗費較少的路徑關係記錄儲存空間，而且只需常數時間查詢表格就能知道任意兩節點間的路徑關係。另外本研究對於圖形結構上任何的更新處理，僅需局部調整非樹邊遞移封包矩陣中相關的路徑關係資訊，而不需要對圖形結構重新編碼。

在未來研究部分，仍有許多議題尚待我們去解決。首先是更新結果的驗證，我們除了證明原有的結構索引正確之外，還必須驗證當結構索引經過新增或是刪除動作後，是否還能確保路徑關係判別的正确性。此外由於本研究所提出的方法在非樹邊端點數甚小於圖形節點數時有較佳的表現，即為 $|V_{nl}|$ 甚小於 $|V|$ 。但是對於 $|V_{nl}|$ 值尚未有明確的界定範圍，因此希望能找到 $|V_{nl}|$ 值的邊界為何，以得知在什麼情況下我們的方法將有最佳的表現。

雖然目前我們所提的方法並未限制選取生成樹的方式與條件為何，但對於非樹邊遞移封包矩陣而言，選取生成樹的方式將決定該矩陣是否為最佳化。若是在取樹的同時能夠決定 $|V_{nl}|$ 的位置，使其盡可能的集中在某些區域，如此將能使非樹邊遞移封包矩陣的使用效能達致最佳化。反之若是 V_{nl} 位置散落於各區域，則非樹邊遞移封包矩陣將可能成為稀疏矩陣 (sparse matrix)，進而造成矩陣空間使用上的浪費。

References

- [1] R. Agrawal, A. Borgida, H. V. Jagadish, "Efficient Management of Transitive Relationships in Large Data and Knowledge Bases," Proceedings of ACM SIGMOD International Conference on Management of Data, 1989.
- [2] H. Ait-Kaci, "An Algebraic-semantic Approach to the Effective Resolution of Types Equations," Theor. Comput. Sci. vol. 45, pp. 185-215, 1986.
- [3] H. Ait-Kaci, R. Boyer, P. Lincoln, R. Nasr, "Efficient Implementation of Lattice Operations," ACM Transactions on Programming Languages and Systems, Vol. 11, No. 1, pp. 115-146, 1989.
- [4] Y. Caseau, "An Object-oriented Deductive Language," Annals of Mathematics and Artificial Intelligence, 1991.
- [5] Y. Caseau, "Efficient Handling of Multiple Inheritance Hierarchies," Proceedings of the International Conference on Object-Oriented Programming Systems, Languages, and Applications, pp. 271-287, 1993.
- [6] E. Cohen, E. Halperin, H. Kaplan, U. Zwick, "Reachability and Distance Queries via 2-hop Labels," Proceedings of the 13th ACM-SIAM SODA, 2002.
- [7] P. F. Dietz, "Maintaining Order in a Linked List," Proceedings of the 14th Annual ACM Symposium on Theory of Computing, 1982.
- [8] A. Fall, "Sparse Term Encoding for Dynamic Taxonomies," Proceedings of the Fourth International Conference on Conceptual Structures, 1996.
- [9] D. Ganguly, C. Mohan, S. Ranka, "A Space-and-time-efficient Coding Algorithm for Lattice Computations," IEEE Transactions on Knowledge and Data Engineering, pp. 819-829, 1994.
- [10] J. Y. Gil, Y. Zibin, "Efficient Subtyping Tests with PQ-Encoding," ACM Transactions on Programming Languages and Systems, Vol. 27, No. 5, pp. 819-856, 2005.
- [11] H. He, H. Wang, J. Yang, P.S. Yu, "Compact Reachability Labeling for Graph-Structured Data," Proceedings of Conference on Information and Knowledge Management, 2005.
- [12] H. V. Jagadish, "A Compression Technique to Materialize Transitive Closure," ACM Transactions on Database Systems, Vol. 15, No. 4, 1990.
- [13] I. Tatarinov, S.D. Viglas, K. Beyer, J. Shanmugasundaram, E. Shekita, C. Zhang, "Storing and Querying Ordered XML Using a Relational Database System," Proceedings of ACM SIGMOD International Conference on Management of Data, 2002.
- [14] H. Wang, R. H. He, J. Yang, P. S. Yu, J. X. Yu, "Dual Labeling: Answering Graph Reachability Queries in Constant Time," Proceedings of the 22nd International Conference on Data Engineering, 2006.
- [15] 李銘桀, 針對多重路徑查詢建構 XML 文件索引機制, 碩士論文, 國立嘉義大學資訊工程研究所, 2002.
- [16] 余美伶, 利用區間索引有效處理 DAG 之路徑可達性查詢, 碩士論文, 國立嘉義大學資訊工程研究所, 2006.